

Computer Security 2026 Security Protocols

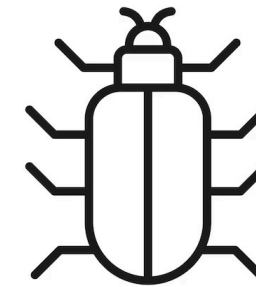
Roberto Guanciale

Security Protocols



To use cryptographic primitives, we need to embed them in a **protocol**: a structured set of messages

- electronic voting
- electronic signature
- authenticated email
- blockchains



Protocol design is subject to subtle errors

Correct protocols allow to remove trust:

With a correct secure messaging protocol, we can distrust the communication channel.

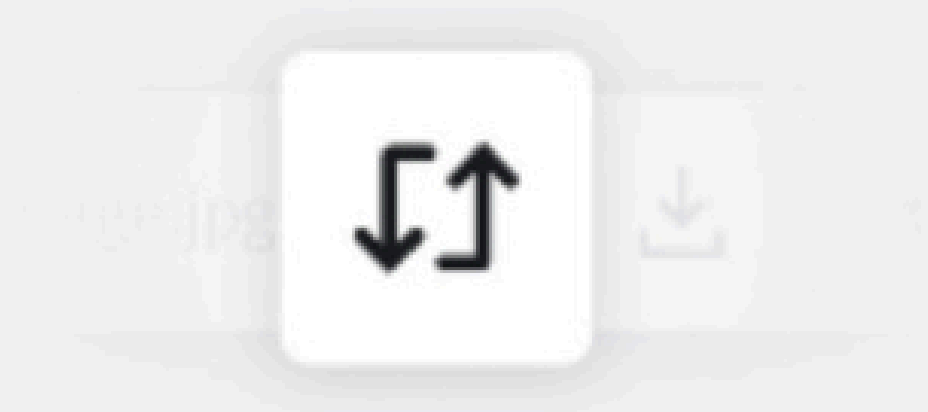


Table of content

- General attacks and principles via simple cases
- 3 real protocols

Toward a secure bank transfer

Alice wants to transfer 100\$
to Bob via the Central bank.

Alice

Central Bank

Alice, Bob, \$100



Toward a secure bank transfer

Alice wants to transfer 100\$
to Bob via the Central bank.

Bob

Central Bank

Alice, Bob, \$200

Bob uses address spoofing



Toward a secure bank transfer

Alice wants to transfer 100\$
to Bob via the Central bank.

Eve

Central Bank

*Alice, **Eve**, \$200*

Eve uses address spoofing



Toward a secure bank transfer

Alice wants to transfer 100\$ to Bob via the Central bank.

Eve eavesdrops the bank transfer

Alice

Central Bank

Alice, Bob, \$100



Eve

Digital Signature via Asymmetric Crypto

Alice wants to sign her message using

H :a secure hash algorithm

E :a public-key encryption algorithm

A_{pk} :public key of Alice

A_{sk} :private key of Alice

Signature: $sign = E_{A_{sk}}(H(msg))$

only owner of A_{sk} could produce the signature!

Verification: is $H(msg) = D_{A_{pk}}(sign)$

In the following we use $\{msg\}_{sk}$

for $msg|E_{sk}(H(msg))$

$msg|E_{sk}(msg)$

Toward a secure bank transfer

Alice wants to transfer 100\$
to Bob via the Central bank.

Alice

Central Bank

$\{Alice, Bob, \$100\}_{A_{sk}}$



Toward a secure bank transfer

Alice wants to transfer 100\$ to Bob via the Central bank.

Alice

Central Bank

$\{Alice, Bob, \$100\}_{A_{sk}}$



Bob can perform a **replay attack**

Bob

$\{Alice, Bob, \$100\}_{A_{sk}}$



$\{Alice, Bob, \$100\}_{A_{sk}}$



$\{Alice, Bob, \$100\}_{A_{sk}}$



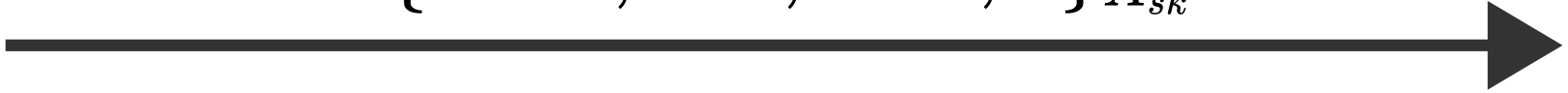
Toward a secure bank transfer

Alice wants to transfer 100\$
to Bob via the Central bank.

Alice

Central Bank

$\{Alice, Bob, \$100, n\}_{A_{sk}}$



Alice uses a nonce **n**, banks
checks that nonce is not in
the Log/DB

Toward a secure bank transfer

Alice wants to transfer 100\$ to Bob via the Central bank.

Alice

Central Bank

$\{Alice, Bob, \$100, t\}_{A_{sk}}$



Alice uses a timestamp **t**,
banks accepts one single
transaction every 5 seconds
and rejects transactions
older than 5 seconds.

Toward a secure bank transfer

Alice wants to transfer 100\$ to Bob via the Central bank

Alice

Central Bank

$\{Alice, Bob, \$100\}_{A_{sk}}$



Lack of recipient in the message means that message can be used for several receivers.

Bob (via replay to different receiver)

Bank 2

$\{Alice, Bob, \$100\}_{A_{sk}}$



Toward a secure bank transfer

Alice wants to transfer 100\$
to Bob via the Central bank

Alice

Central Bank

$\{Alice, Bob, \$100, CB\}_{A_{sk}}$



Bank 2

Toward a secure bank transfer

Alice wants to transfer 100\$
to Bob via the Central bank

Alice

Central Bank

$\{Alice, Bob, \$100, CB, t\}_{A_{sk}}$



Lack of acknowledge means
Alice does not know if the
message has been received
(could have been dropped or
rejected by the bank due to
being altered)

Toward a secure bank transfer

Alice wants to transfer 100\$
to Bob via the Central bank

Alice

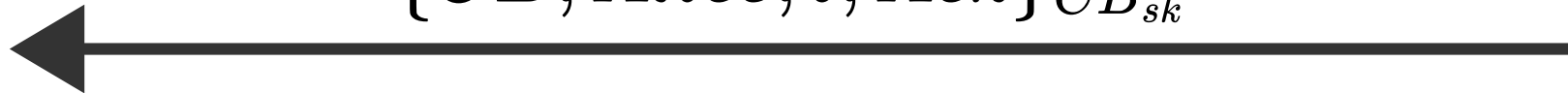
Central Bank

$\{Alice, Bob, \$100, CB, t\}_{A_{sk}}$



Lack of acknowledge means
Alice does not know if the
message has been received
(could have been dropped or
rejected by the bank due to
being altered)

$\{CB, Alice, t, Ack\}_{CB_{sk}}$



Common Attacks

Known-key attacks. An adversary obtains some key used previously and then uses this information to determine new keys

Replay attacks. An adversary records parts of a session and replays them later

Impersonation attacks. An adversary impersonates one of the other principals in the protocol (Person in the middle)

Reflection attacks. An adversary records parts of a session and replays it to the party that originally sent it (usually when shared keys are used)

Desired properties

Confidentiality, authentication, integrity

Freshness. Does this message belong to this instance of this protocol, or is it a replay from a previous run of this protocol?

Explicitness. Is this message really a member of this run of the protocol, or is it copied from a run of another protocol with an entirely different function and different participants?

Forward secrecy. If a key is compromised, do confidential information communicated in the past stay confidential? (e.g. leak new session keys should not compromise previous messages)

Guidelines

Ensure that each message contains a nonce, and require that a reply to a message includes the nonce of the message being replied to, as well as its own new nonce value.

Ensure that each message explicitly contains the name of the sender and intended recipient.

Ensure that each message specifies the security protocol, the version number of that protocol, and the message number within this instance of that protocol.

Toward a secure bank transfer

Alice wants to transfer 100\$
to Bob via the Central bank

Alice

Central Bank

$\{Alice, Bob, \$100, CB, t\}_{A_{sk}}$



How can Central Bank learn A_{pk}
if it does not know it?

Digital Certificates



A **certificate** certifies authenticity:
a public key (and other information)
belongs to a subject

certificates may contain additional info:
validity conditions, authorization,
timestamps

A certificate is issued by **certificate authority** (CA):

CA certifies that Alice's key is A_{pk}

$$cert_A = \{Alice, A_{pk}\}_{CA_{sk}}$$

Requirements:

Central Bank trusts CA

Central Bank knows CA_{pk}

CA knows A_{pk}

verify certificate:

$$H(Alice, A_{pk}) = D_{CA_{pk}}(cert_A. signature)$$

Digital Certificates: chain of trust



Confirm authenticity of CA1 with certificate by another CA2

Example:

$\{Alice, A_{pk}\}_{CA1_{sk}}$ CA1 certifies that Alice's key

$\{CA1, CA1_{pk}\}_{CA2_{sk}}$ CA2 certifies that CA1's key

...

root certificates are (blindly) trusted. These are self-signed.

See root chain on Chrome

Digital Certificates: chain of trust



Certificate Viewer: ns-vip-14.sys.kth.se



General

Details

Certificate Hierarchy

▼ USERTrust ECC Certification Authority

▼ GEANT OV ECC CA 4

ns-vip-14.sys.kth.se

Digital Certificates: Web of Trust

Decentralised alternative to certificate chains

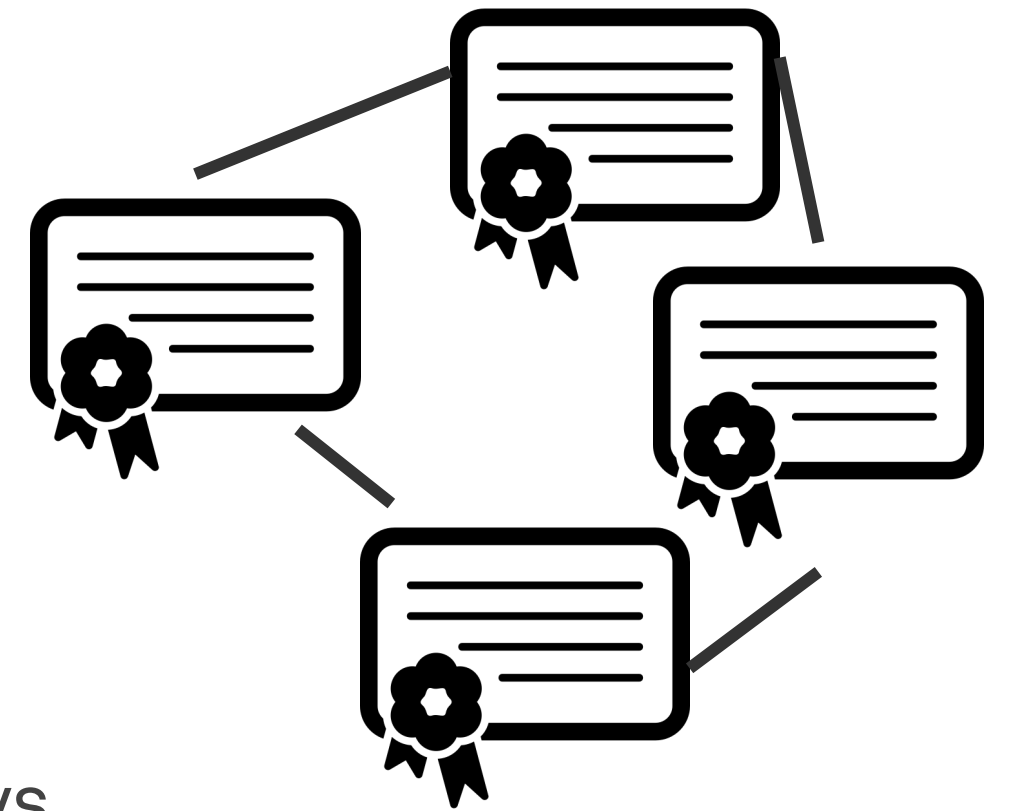
Each user has an asymmetric key pair

Each user has a key ring that holds other user's public keys

If Alice trusts Bob, she signs Bob's public key: $\{Bob, Bob_{pk}\}_{Alice_{sk}}$

With deferred trust, if Charlie trusts Alice, then they can decide to also trust Bob (after verifying the signature $\{Bob, Bob_{pk}\}_{Alice_{sk}}$ using $Alice_{pk}$)

Individual users sign keys of users that they trust.



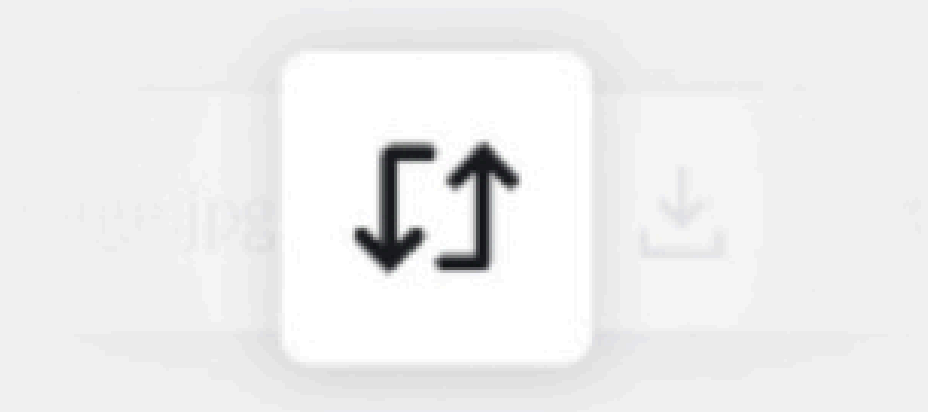


Table of content

- Key Exchange
- WPA vulnerabilities
- TLS

Session Key exchange

It is **expensive** to use public encryption

Two parties negotiate a (symmetric) **shared-secret**, based on public keys.

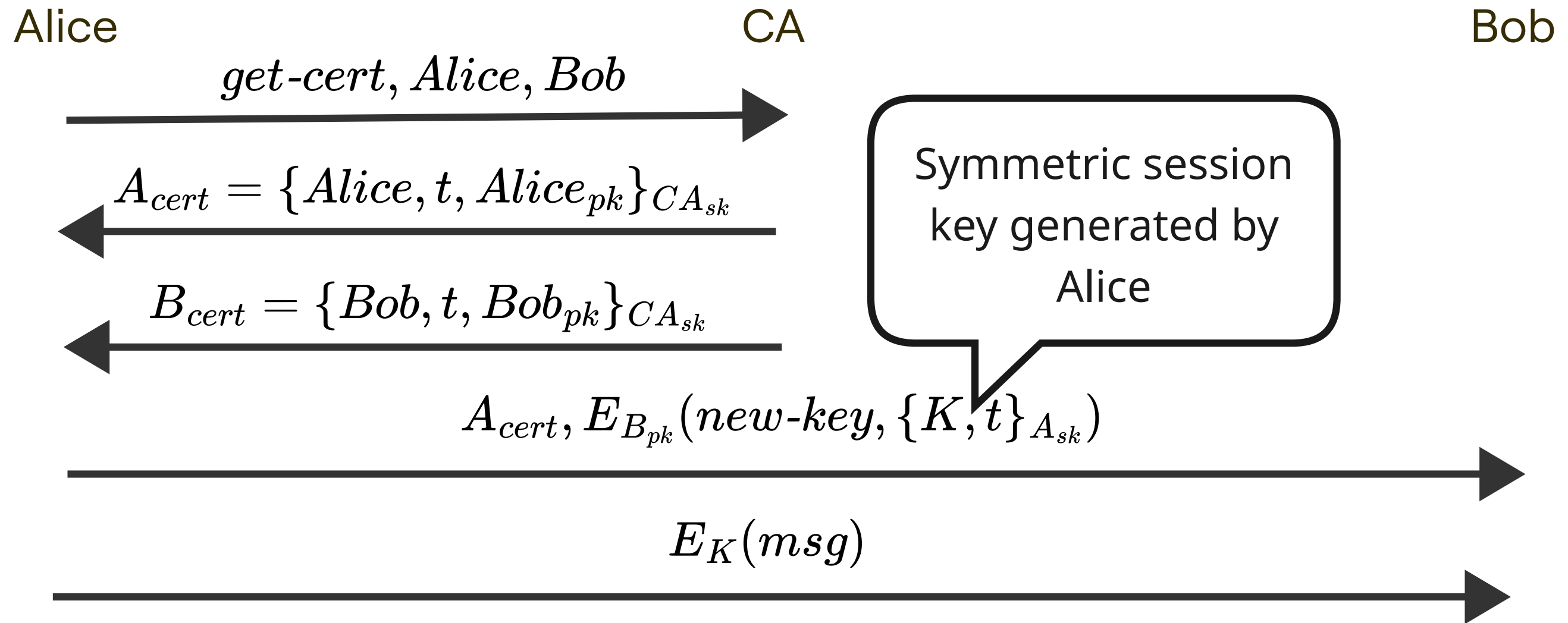
Entities: Alice, Bob, Certificate Authority (CA)

- Alice trusts CA (has/knows CA_{pk})
- Bob trusts CA (has/knows CA_{pk})

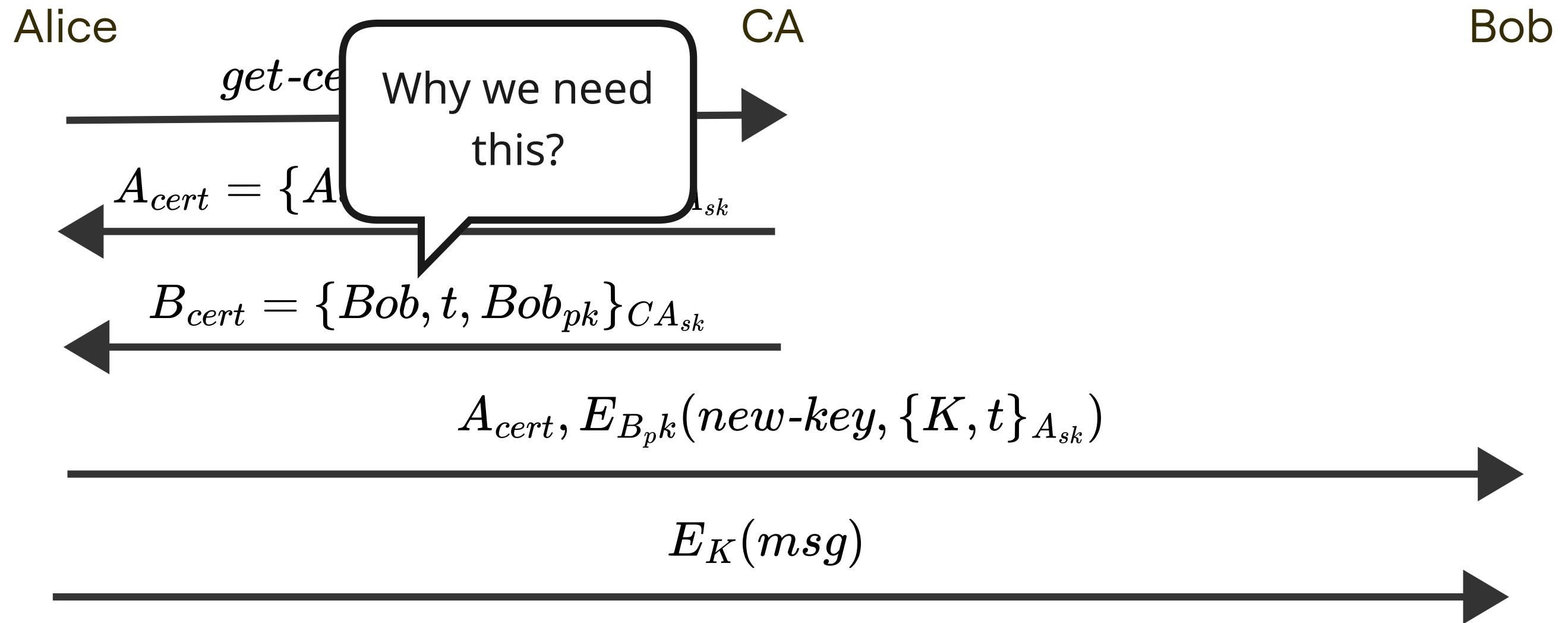
two versions:

incorrect (by Denning and Sacco [DS81]) and
fixed protocol

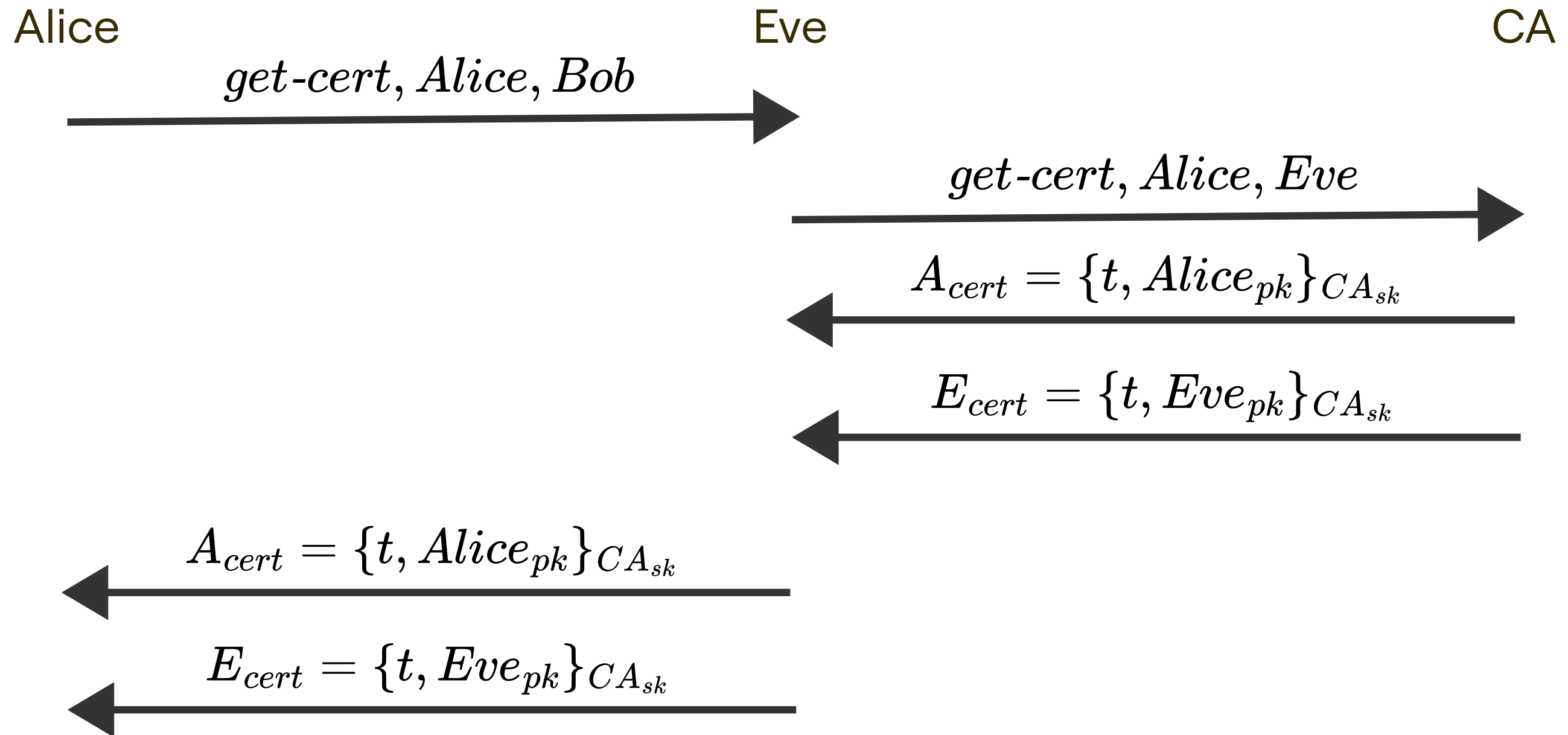
Denning and Sacco



Denning and Sacco



Reflection Attack



Denning and Sacco

Alice

CA

Bob

$get-cert, Alice, Bob$

$A_{cert} = \{Alice, t, Alice_{pk}\}_{CA_{sk}}$

$B_{cert} = \{Bob, t, Bob_{pk}\}_{CA_{sk}}$

$A_{cert}, E_{B_{pk}}(new-key, \{K, t\}_{A_{sk}})$

$E_K(msg)$

Why we need
this?

Message reuse attack

In 1994, Abadi and Needham found a flaw in the Denning-Sacco protocol: $\{K, t\}_{A_{sk}}$ can be reused by Bob. Bob can **impersonate** Alice.

Alice

Bob

Eve

$A_{cert}, E_{B_{pk}}(new-key, \{K, t\}_{A_{sk}})$



$A_{cert}, E_{E_{pk}}(new-key, \{K, t\}_{A_{sk}})$



Message reuse attack

In 1994, Abadi and Needham found a flaw in the Denning-Sacco protocol: $\{K, t\}_{A_{sk}}$ can be reused by Bob. Bob can **impersonate** Alice.

Alice

Bob

Eve

$A_{cert}, E_{B_{pk}}(new-key, \{Alice, Bob, K, t\}_{A_{sk}})$



Key derivation attack

Bob may infer that $K+1$ is used as session key with Eve

Alice

Bob

Eve

$A_{cert}, E_{B_{pk}}(new-key, \{Alice, Bob, K, t\}_{A_{sk}})$



$A_{cert}, E_{E_{pk}}(new-key, \{Alice, Eve, K + 1, t_1\}_{A_{sk}})$



Compromised forward secrecy

If a Key is leaked, all previous messages are compromised

Alice

Bob

$A_{cert}, E_{B_{pk}}(new-key, \{Alice, Bob, H(H(rnd)), t\}_{A_{sk}})$



$A_{cert}, E_{B_{pk}}(new-key, \{Alice, Bob, H(rnd), t_1\}_{A_{sk}})$

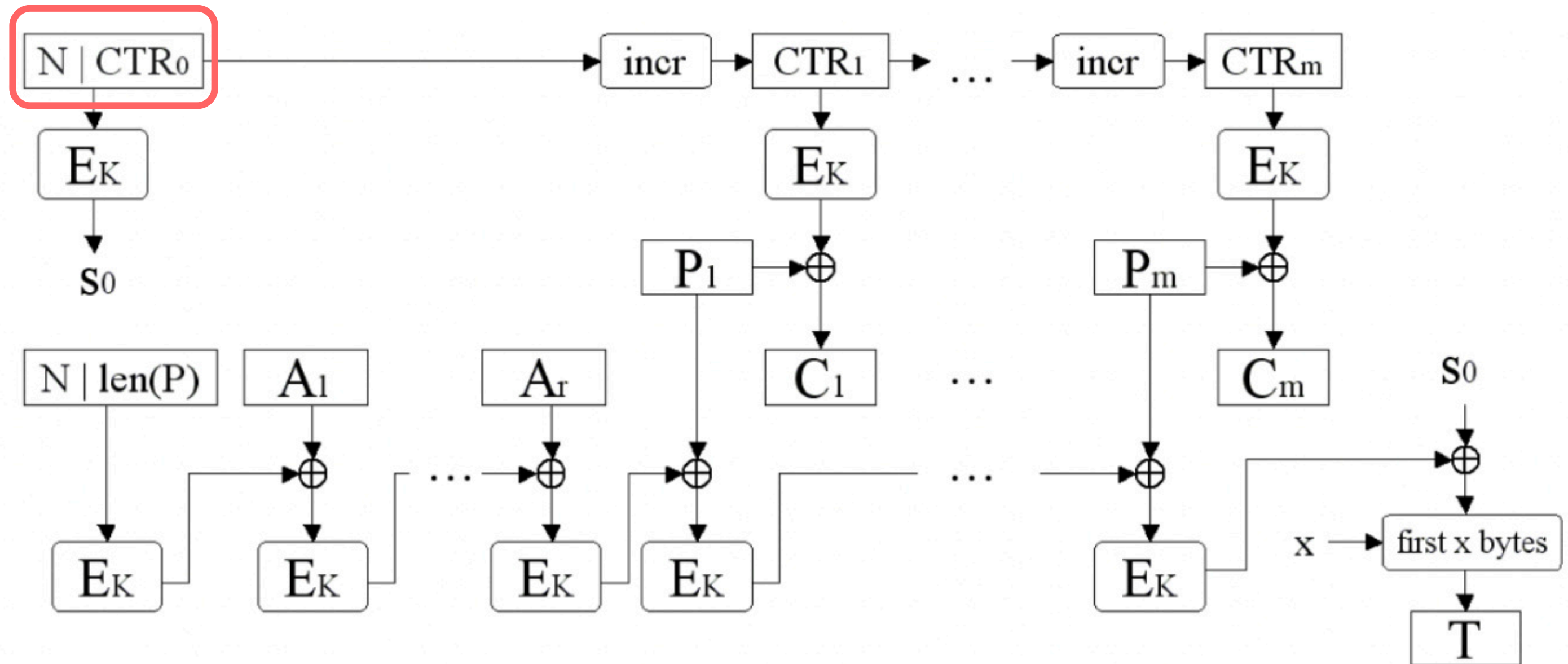


$A_{cert}, E_{B_{pk}}(new-key, \{Alice, Bob, rnd, t_2\}_{A_{sk}})$



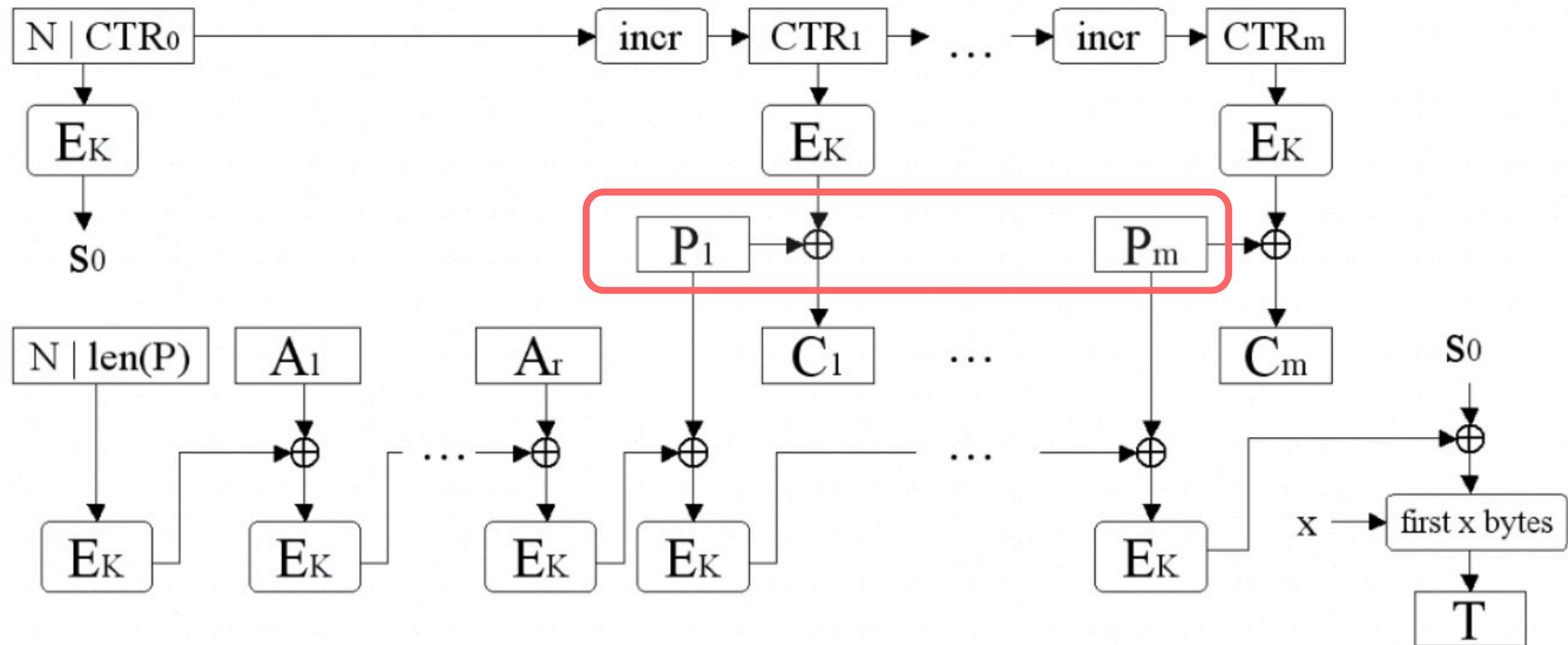
WPA2 AES-CCMP

Nonce: packet number, MAC, etc



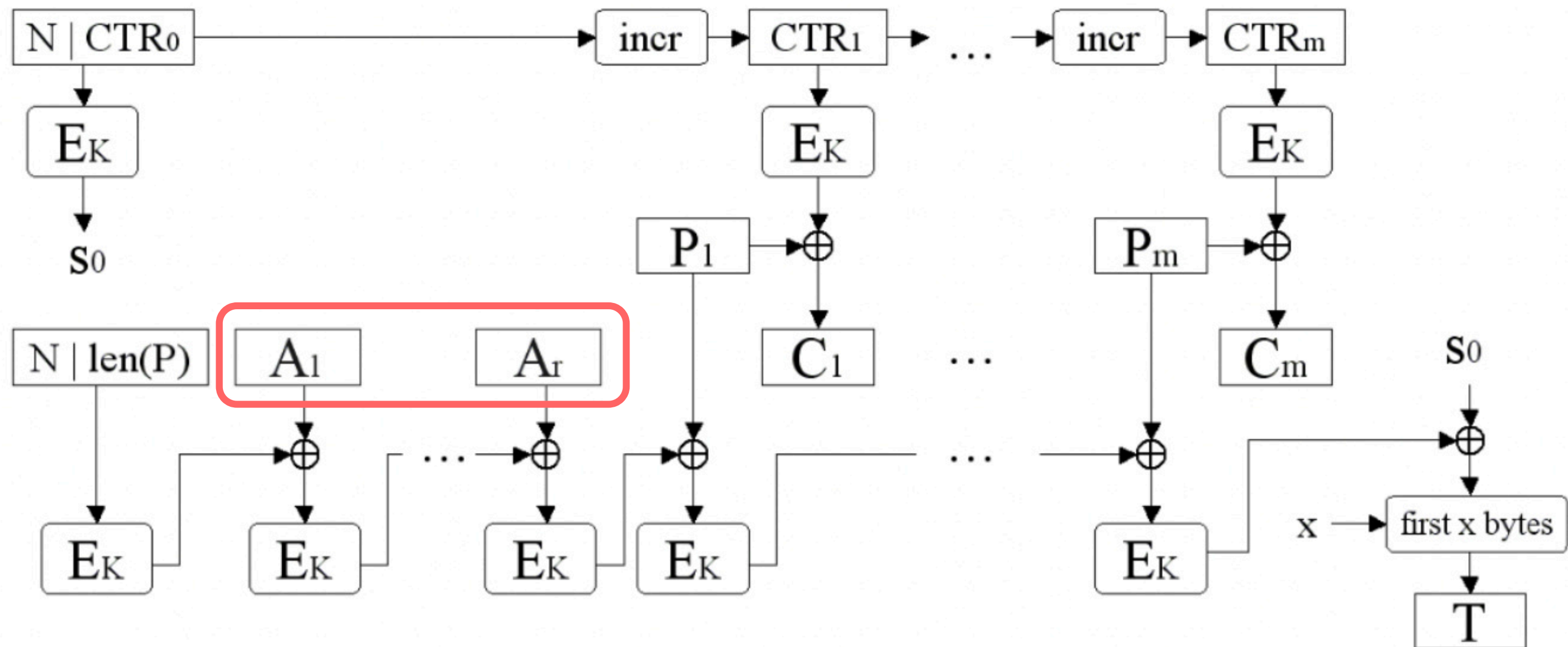
WPA2 AES-CCMP

Plaintext blocks to be transmitted



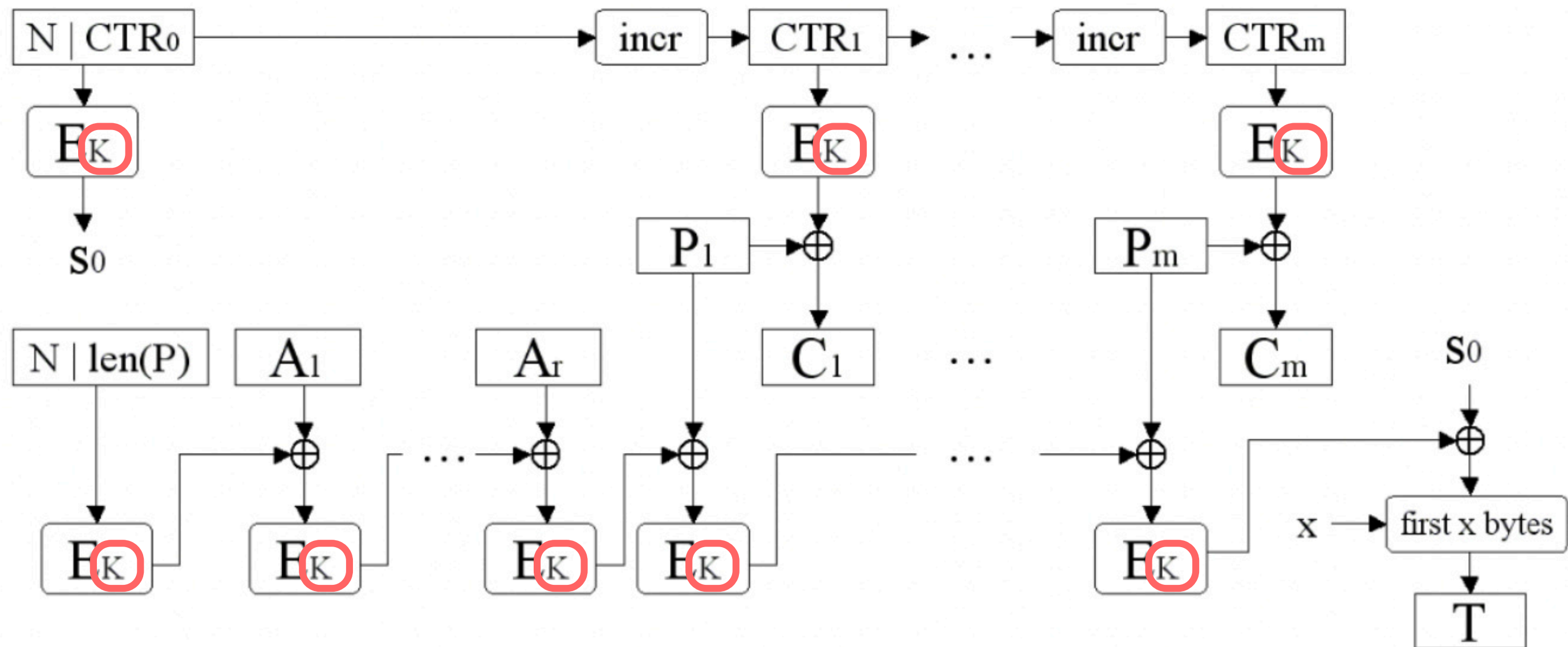
WPA2 AES-CCMP

Additional data (i.e., WiFi headers) to be authenticated but not encrypted



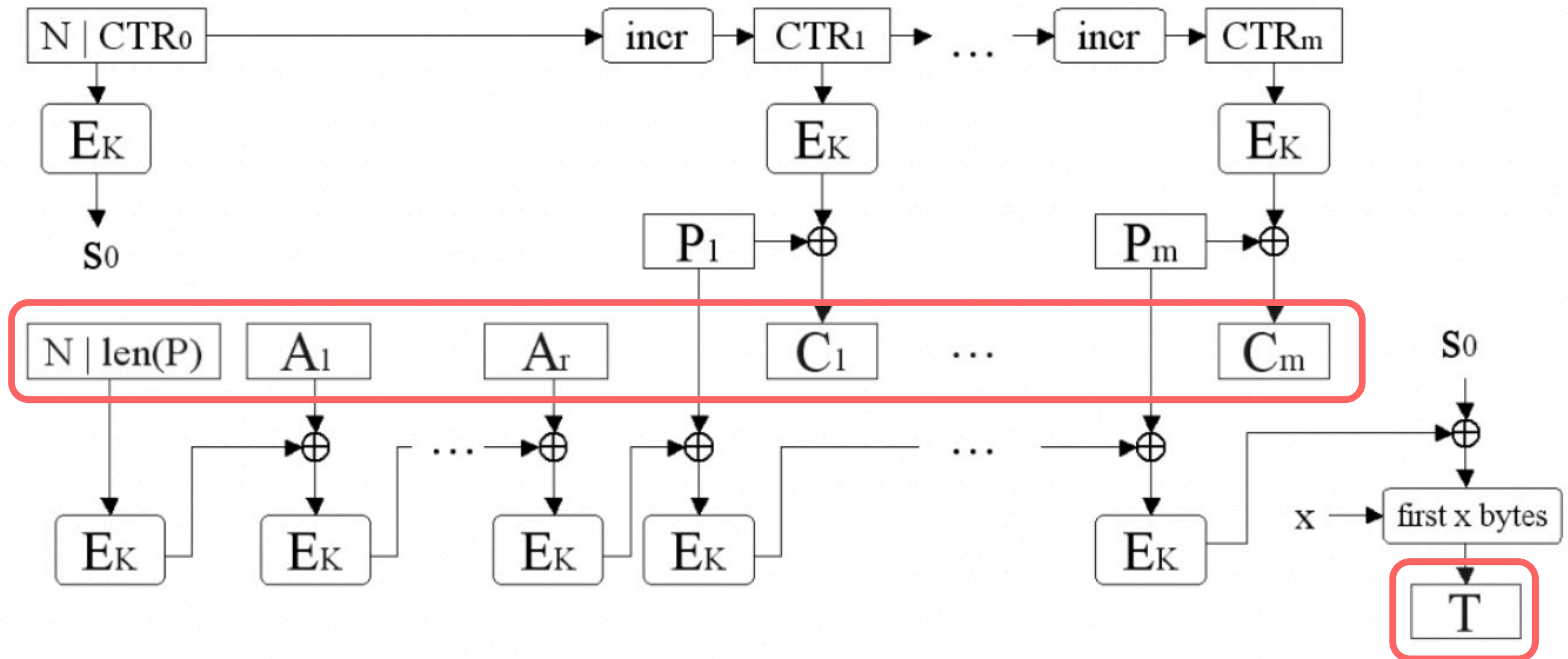
WPA2 AES-CCMP

AES Key shared between client and access point



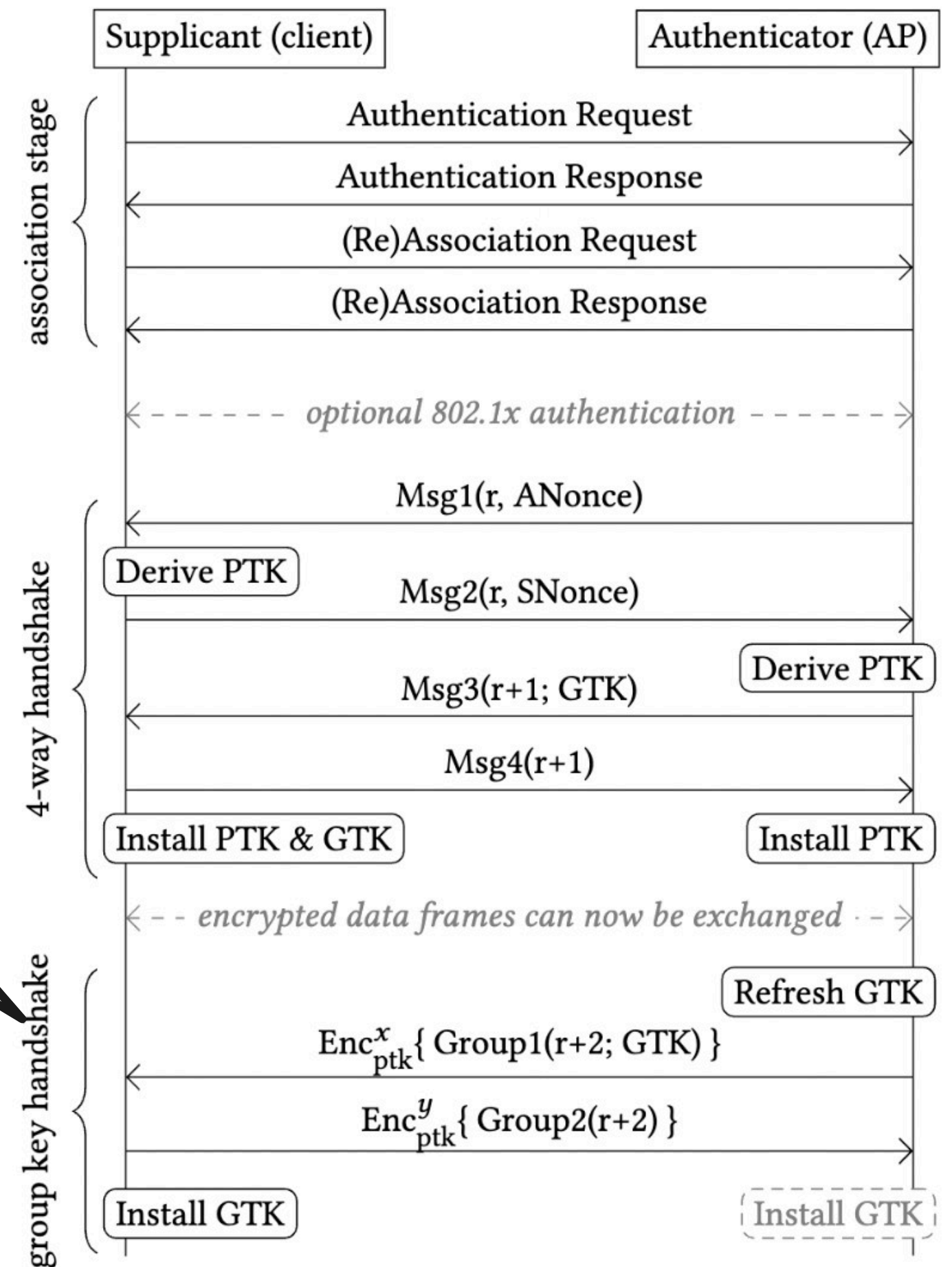
WPA2 AES-CCMP

Transmitted frames, including MAC



WPA2 AES-CCMP

Reset counter

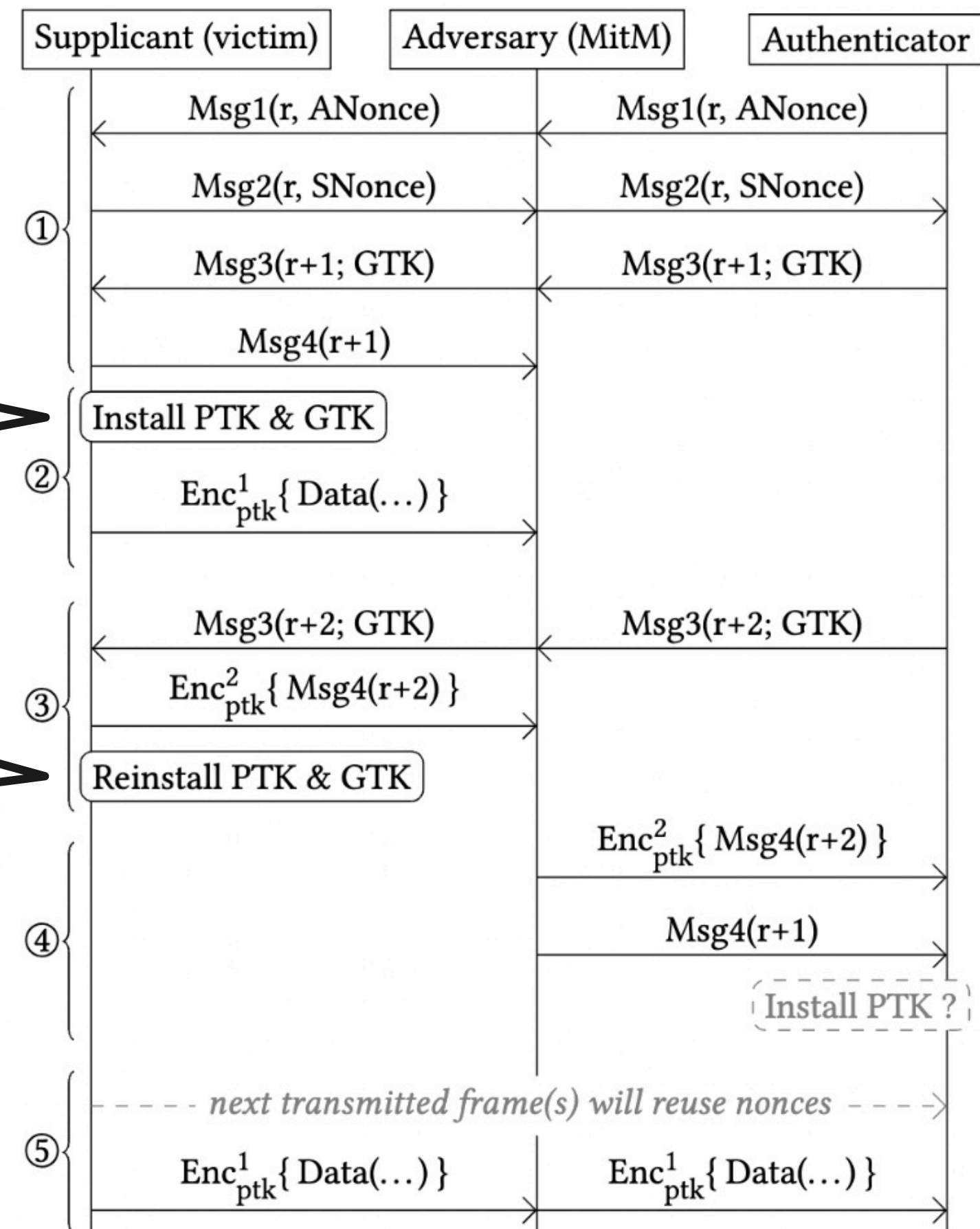


WPA2 AES-CCMP

KRACK Attack

Reset counter

Reset counter



Transport Layer Security (TLS)

confidentiality: sent data is kept secret

integrity: attackers cannot modify traffic

authenticity of server: client is communicating with the legitimate server

relies on in-order delivery of messages

by TCP = stateful network communication protocol between two parties

Used in various protocols:

- Simple Mail Transport Protocol SMTP (using STARTTLS) for email delivery
- Virtual Private Network (VPN) encrypt traffic between user and a server
- HTTPS = HTTP + TLS

TLS Handshake (1-2)

Assume an underlying TCP connection has already been established

1. The client sends *ClientHello* with
 - a. A 256-bit random number **RB** (“client random”)
 - b. A list of supported cryptographic algorithms

2. The server sends *ServerHello* with
 - a. A 256-bit random number **RS** (“server random”)
 - b. The algorithms to use (chosen from the client’s list)
 - c. Its digital **certificate**, which contains the server’s public key, signed by a trusted Certificate Authority.

RB and **RS** prevent replay attacks

RB and RS are randomly chosen for every handshake

This guarantees that two handshakes will never be exactly identical

The client validates the **certificate**

The client now knows the server’s public key

Certificates are public. Anyone can provide a valid certificate

The client is not yet sure that they are talking to the legitimate server

TLS Premaster Secret (3) [e.g. RSA]

This step has **two main purposes**

- (1) Make sure the client is talking to the legitimate server (not an impersonator)
The server must prove that it owns the private key corresponding to the public key in the certificate
- (2) Give the client and server a shared secret

The client randomly generates a premaster secret (**PS**)

The client encrypts PS with the server's public key and sends it to the server

The client knows the server's public key from the certificate

The server decrypts the premaster secret

Nobody except the legitimate server can decrypt the PS:

Proves that the server owns the private key

The client and server now share a secret

TLS Derive Key (3) [e.g. RSA]

The server and client each derive symmetric keys from **RB**, **RS**, and **PS**

CB: For encrypting client-to-server messages

CS: For encrypting server-to-client messages

IB: For MAC-ing client-to-server messages

IS: For MAC-ing server-to-client messages

Note: Both client and server know all four keys

TLS – MACs (4)

The server and client exchange MACs on all the messages of the handshake so far

Recall MACs: Any tampering on the handshake will be detected

TLS – Messages (5)

Messages can now be sent securely Encrypted and MAC'd

TLS – Properties

How can we be sure we are talking to the legitimate server?

The server sent its certificate and client validated it, so we know the server's public key

The server proved that it owns the corresponding private key

In case of RSA: The server decrypted the **PS**

An attacker impersonating the server would not have the server's private key (assuming they have not compromised the server)

TLS – Properties

How can we be sure that network attackers can't read or tamper with our messages?

The attacker doesn't know **PS**

PS was encrypted with the server's public key

The symmetric keys are derived from **PS**

The attacker doesn't know the symmetric keys used to encrypt and MAC messages

Encryption and MACs provide **confidentiality** and **integrity**

TLS – Properties

How can we be sure that the attacker hasn't replayed old messages from a past TLS connection?

Every handshake uses a different **RB** and **RS**

The symmetric keys are derived from **RB** and **RS**

The symmetric keys are different for every connection

TLS – Downgrade of cipher suite

Export-grade ciphers are weak ciphers

U.S. export regulations from 1990 had forbidden export of strong ciphers
e. g. RSA with 512 bit key

P(M)ITM attack:

- attacker downgrades ciphersuites to export-grade ciphers
- solve the hard problems (Discrete Logarithm Problem, or RSA Problem)
- record messages and recover encrypted communication (passwords, ...)

8.5% of top 1 million websites supported export-grade RSA in SSL/TLS (in 2015)

TLS – Open SSL Heartbleed

We will see in memory safety

TLS – Apple Bug

```
46
47     hashOut.data = hashes + SSL_MD5_DIGEST_LEN;
48     hashOut.length = SSL_SHA1_DIGEST_LEN;
49 -   if ((err = SSLFreeBuffer(&hashCtx, ctx)) != 0)
50 +   if ((err = SSLFreeBuffer(&hashCtx)) != 0)
51         goto fail;
52
53 -   if ((err = ReadyHash(&SSLHashSHA1, &hashCtx, ctx)) != 0)
54 +   if ((err = ReadyHash(&SSLHashSHA1, &hashCtx)) != 0)
55         goto fail;
56     if ((err = SSLHashSHA1.update(&hashCtx, &clientRandom)) != 0)
57         goto fail;
58     if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
59         goto fail;
60     if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
61         goto fail;
62 +   goto fail;
63     if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
64         goto fail;
65
66     err = sslRawVerify(ctx,
67                       ctx->peerPubKey,
68                       dataToSign,                /* plaintext */
69                       dataToSignLen,             /* plaintext length */
70                       signature,
71                       signatureLen);
72     if(err) {
73         sslErrorLog("SSLDecodeSignedServerKeyExchange: sslRawVerify -
74                     "returned %d\n", (int)err);
75         goto fail;
76     }
77
78 fail:
79 -   SSLFreeBuffer(&signedHashes, ctx);
80 -   SSLFreeBuffer(&hashCtx, ctx);
81 +   SSLFreeBuffer(&signedHashes);
82 +   SSLFreeBuffer(&hashCtx);
83     return err;
84
85 }
86
```

An additional goto statement causes remaining checks to be skipped.

Caused iOS and Mac devices to accept invalid certificates in Apple's SSL/TLS implementation.

Threat to **authenticity**:
Impersonation attacks were possible

Formal verification of protocols

Usually protocols are verified w.r.t. ideal models and required that probability for attacker to distinguish the two protocols is negligible

Tamarin symbolic models of protocols:

- cryptographic operations are abstracted

- attackers are modeled with capabilities such as intercepting and modifying communications.

- e.g. may miss some vulnerabilities if one does not state explicitly that nonces should not be reused

Easy Crypt Computational model.

- Probabilistic reasoning and reductions to demonstrate security, capturing actual algorithmic weaknesses, including those based on computational hardness assumptions.